CS 739 MadKV Project 2

Group members: Fariha Tabassum Islam `fislam2@wisc.edu`, Anusha Kabber `kabber@wisc.edu`

Design Walkthrough

Our project is divided into `kvserver`, `kvclient`, `kvstore` and `kvmanager`

Following is brief description of our design:

Server:

The server integrates tightly with the Raft consensus algorithm to ensure linearizable consistency across replicas within a partition. Each server instance is part of a specific Raft group (defined by part_id and rep_id) and is backed by a RaftKVStore that mediates all reads and writes through the local Raft node. All client-modifying operations like Put, Swap, and Delete are serialized via an internal request lock and submitted to the Raft log, ensuring they are replicated and applied in a consistent order across all followers. The server only services requests for keys belonging to its partition, and relies on Raft's leader election and log replication mechanisms to maintain availability and correctness in the face of node failures or network partitions. Also, we used hash based partition.

Storage backend:

The notable storage backend in this server is a RaftKVStore. Rather than applying operations directly to a local database, write requests are first appended to the Raft log via the RaftNode, replicated to a majority of replicas, and only then applied to the underlying key-value store once committed. This design ensures that all replicas apply the same operations in the same order, a core requirement for consensus. The backend is durable — writes are persisted after consensus is reached, preventing data loss even if the leader fails. By coupling Raft log application with state machine updates (the key-value store), the backend becomes a reliable state replication layer, providing the linearizable semantics Raft guarantees.

Client:

The client first fetches the partition-to-replica mappings from the cluster manager and maintains a local cache of leader addresses for each partition. Before issuing any request (e.g., PUT, GET, DELETE), the client hashes the key to determine its partition, checks or discovers the current leader by pinging replicas (via a special **ping** key), and sends the request to that leader. If the leader responds with a NotLeader error and the new leader's ID is embedded in the message, the client updates its cache and retries the operation—ensuring correctness under Raft's leader election dynamics. This design allows the client to respect Raft's

single-writer semantics per partition and guarantees strong consistency by never writing to followers.

Cluster manager: As specified, clients and servers query the manager. Upon start server register to manager and are assigned a few range partitions. Upon start, clients query manager determine partition mappings. This manager module is designed to coordinate the partitioning and registration of Raft-based replica groups in a distributed key-value store. Each partition is associated with a set of server replicas, forming independent Raft consensus groups to ensure fault-tolerant replication. The manager tracks registration status for each server in every partition, and only provides partition-to-replica mappings to clients once all expected replicas are registered—ensuring Raft groups are fully formed before operation.

Error Handling: As specified, upon failure, indefinite retry is done.

Testcases

Run test1_p3.sh under test_scripts to test scenario 1 and 2 in Custom testing.

For Scenario 1 (kill follower):

bash test1_p3.sh

This script launches a replicated key-value store with: 1 manager, 2 partitions, 3 replicas per partition (i.e., f = 1 tolerance in Raft-style quorum). In a separate terminal, run client:

cargo run –bin kvclient – –manager_addrs 127.0.0.1:3666 –verbose

Try some client commands.

You will be prompted by the test script to manually specify one follower replica to kill from each partition. Kill the replicas and re test client commands. We see the cluster working as expected.

For Scenario 2 (kill leader):

Similarly prompted to kill a replica. Choose a leader to kill. We again see the cluster working as expected after initial failed attempts: it retried multiple times (5+ by default) before giving up and going into leader detection mode. Pinged a different replica, got redirected to new leader. Client commands after this process continued to give consistent and expected results.

For Scenario 3 (kill mulitple followers):

bash test2_p3.sh

Simultaneously run:

cargo run –bin kvclient – –manager_addrs 127.0.0.1:3666 –verbose

Try some client commands.

You will be prompted by the test script to manually specify replicas to kill from a partition. Choose the partition that your key is in and kill a majority of followers.

Restart client. Try to get the same key. You should get a hanging request. Here too, we see expected performance.

Explanations

We conducted three custom test scenarios using provided scripts to validate system behavior under failure. In Scenario 1, we killed one follower in each partition (<= f), and the system continued operating normally, with client requests succeeding and data consistency maintained. In Scenario 2, we killed a leader replica; the client initially encountered failures, retried several times, detected the new leader, and resumed successful operations—demonstrating correct leader recovery behavior. In Scenario 3, we killed more than f replicas in a partition, and the client request for a key in that partition hung as expected, confirming the system's correct transition to unavailability when quorum was lost.

Fuzz Testing

Fuzz testing passed with and without crash.

server_rf	crashing	outcome
5	no	PASSED
5	yes	PASSED

Scenario 1: Run testfuzz_p3.sh. Fuzz testing PASSED.

Scenario 2: Run testfuzz_p3.sh. Run crashfuzz_p3.sh while fuzz testing is running. Fuzz testing PASSED.

Comments

As instructed, Scenario 1: No Failures Launched a cluster with 1 partition and server replication factor of 5.

Ran the fuzzer without crashing any servers.

Result: The system completed fuzz testing successfully with no inconsistencies or crashes.

Scenario 2: <= f Failures Launched the same cluster with RF = 5.

Manually crashed two replicas in the partition, including the current leader.

Fuzzer was run concurrently and completed successfully.

Result: The system recovered from the leader failure, maintained quorum, and remained available and consistent.

Outcome: Both test scenarios completed successfully. The system passed all fuzzing trials without abnormality.

YCSB Benchmarking

10 clients throughput/latency across workloads & number of partitions:

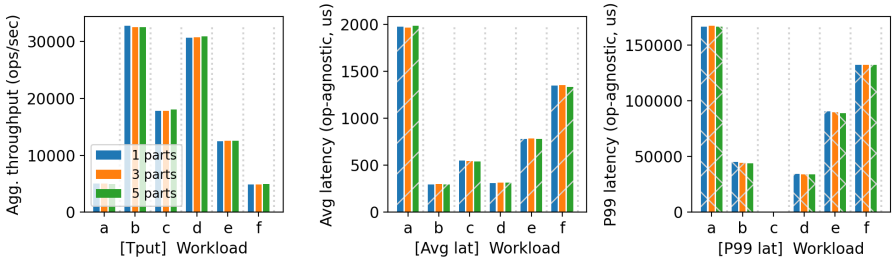

Figure 1: ten-clients

Agg. throughput trend vs. number of clients w/ and w/o partitioning:

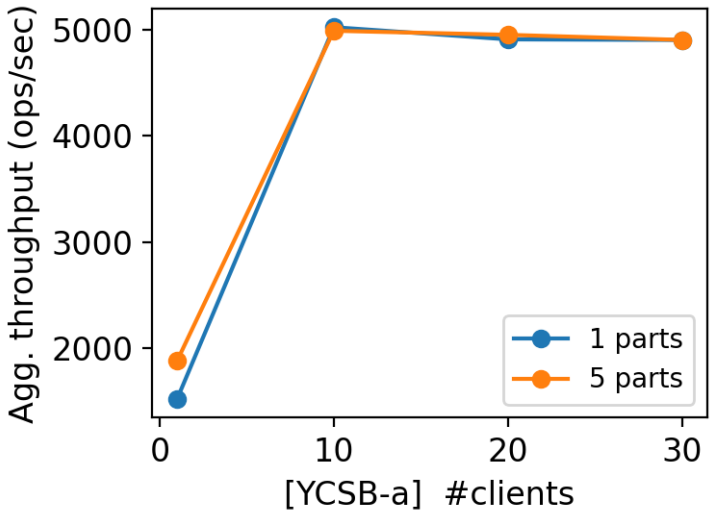

Figure 2: tput-trend

Comments

We run our experiments on `cloudlab` machine type `c220g5`, which has 2 sockets, 10 CPU core per socket and 2 threads per core, therefore total 40 vCores. The cpu model is Intel(R) Xeon(R) Silver 4114 CPU @ 2.20GHz. It has 187GB memory. We ran all of the servers and clients in the same machine.

Following is the summary of the YCSB workload operations ratio, which helps to explain the results. The figures are from our state machine and log implementation.

Workload	Operation1	Operation2
A	READ 50%	UPDATE 50%
B	READ 95%	UPDATE 5%
C	READ 100%	
D	READ 95%	INSERT 5%
E	SCAN 95%	INSERT 5%
F	UPDATE 50%	READ 100%

We could not complete running the experiments, therefore result not included.